

AirSense — 14-Day Execution Plan (v2: Nodal-Officer-First)

Theme 2: Rising AQI Levels in Modern Urban Cities

Team size: 4 | Primary user: the nodal officer, not the citizen

0. Why the Reframe Matters (read this first, whole team)

Your own SAMEER notes describe the *actual* bottleneck, and it isn't citizen awareness — citizens already file thousands of complaints. The bottleneck is what happens **after** a complaint lands on a nodal officer's desk:

- They claim a 24-hour SLA but complainants get no clarity on *who* is actually solving it.
- Geotag-based routing exists, but **problem categorization is still asked of the user manually** — there's no automated clustering of what's actually happening.
- No dedup: the same construction-dust event reported by 40 residents in one neighborhood likely shows up as 40 separate tickets, not one prioritized incident.
- No systematic priority logic — everything queues roughly FIFO, with no equivalent of "an electricity outage matters more right now than a 4-hour water stoppage."
- Even the *data itself* is inconsistent (SAMEER app vs. website showing different AQI readings), which undermines an officer's ability to trust what they're triaging.

New one-line pitch:

"AirSense doesn't replace the citizen complaint channel — it sits behind it, doing in seconds what a nodal officer currently does manually over hours: clustering duplicate reports, ranking them by real urgency, and handing back a ready-to-act recommendation instead of a raw complaint list."

This is a **B2G ops-efficiency tool**, not a citizen-facing app. Every screen you build should answer: *does this save a nodal officer time or improve the quality of their decision?* If a feature doesn't do one of those two things, cut it.

1. Team Roles (same 4 people, re-scoped ownership)

Role	Owns	Key reference
R1 — ML/Data Lead	Clustering (dedup + areawise/typewise grouping), priority-scoring model, AQI trend model feeding the "why now" justification	AirSense_ML_Method_Recommendations.docx , ML_Methods_by_Data_Type_AQI.docx
R2 — Backend/Geo Lead	FastAPI ingestion pipeline, hotspot geocoding, priority queue endpoint, automated-recommendation trigger to Gemini, before/after resolution tracking	AirSense_Tools_Requirements_Proposal.pdf
R3 — Frontend Lead	The nodal officer dashboard : hotspot map, prioritized queue, cluster detail view, one-click "generate recommendation," resolve + outcome logging	Existing AirSense components (repurpose Dashboard/InteractiveMap/CommunityFeed logic — different audience, same underlying architecture)
R4 — Research/Pitch Lead	Quantify officer time-savings, economic viability, environmental impact, competitive positioning, deck + demo video	AirSense_Competitive_Landscape_Analysis.md , SAMEER complaint notes

Daily 15-min standup, same as before — non-negotiable.

2. Scope Decision (lock Day 1)

Build a nodal-officer triage dashboard with five features, in this exact priority order:

1. **Segregation by city/location** — ingest synthetic complaints (Faker.py + a Kaggle AQI/complaints dataset, per your own notes — you will not get real SAMEER/CPCB access in 14 days) tagged with geotag + category.
2. **Clustering (dedup + areawise/typewise grouping)** — DBSCAN or KMeans on (lat, lng, category, time-window) so 40 duplicate reports collapse into 1 actionable incident. This is your single highest-leverage feature — it's the one manual step your notes explicitly call out ("asking users itself about problem category").

- Hotspot + priority ranking** — a weighted score (complaint volume in cluster + category severity + time open + AQI reading in that zone) replacing FIFO with actual urgency ordering.
- Automated recommendation** — a pre-templated prompt (swap in cluster's data: location, category, severity, AQI trend) sent to Gemini/Claude, returning a ready-to-review action recommendation instead of a blank ticket.
- Outcome tracking** — officer marks a cluster "actioned," logs before/after AQI for that zone. This is what turns a triage tool into an evidence-generating one over time.

Explicitly out of scope for 14 days: citizen-facing complaint app polish, real CPCB/SAMEER API integration, LSTM/CNN, live sensor hardware. Mention these as later phases in the deck; don't build them.

3. Day-by-Day Plan

Phase A — Foundation (Days 1-2)

Day	R1 (ML)	R2 (Backend/Geo)	R3 (Frontend)	R4 (Research/Pitch)
1	Design synthetic complaint schema with Faker.py: <code>complaint_id</code> , <code>lat</code> , <code>lng</code> , <code>category</code> , <code>timestamp</code> , <code>description</code> , <code>reported_AQI</code> , <code>status</code> . Source a Kaggle India AQI dataset to backfill realistic AQI values per area	FastAPI skeleton + SQLite; define <code>complaints</code> , <code>clusters</code> , <code>actions</code> tables	Audit existing components for reuse — InteractiveMap → hotspot map, CommunityFeed → complaint list logic	Lock the reframe (Section 0) with the whole team; draft the "manual process today vs. AirSense" before/after slide skeleton
2	Generate 500-1000 synthetic complaints across ~5-8 Pune zones, several categories (construction dust, vehicular, biomass burning, industrial), clustered in realistic hotspots (don't uniformly randomize — bake in 2-3 obvious hotspots so your demo clustering actually looks meaningful)	Build <code>/complaints/ingest</code> + <code>/complaints/list</code> endpoint	Wireframe the officer dashboard in Figma: map view + priority queue + cluster detail panel	Draft time-savings claim: estimate manual triage time per complaint today vs. projected time with clustering+auto-recommendation (state your assumptions explicitly — judges respect a stated assumption more than a vague "saves time")

Phase B — Core Intelligence (Days 3-6)

Day	R1 (ML)	R2 (Backend/Geo)	R3 (Frontend)	R4 (Research/Pitch)
3	Build clustering pipeline (DBSCAN on lat/lng + category, time-windowed) — this collapses duplicate reports into one incident	Build <code>/clusters</code> endpoint serving R1's grouped output	Build the hotspot map, rendering clusters as sized/colored markers (size = complaint count)	Feasibility section: sensor/data-quality caveats — including the SAMEER app-vs-website AQI mismatch your own notes flagged, and how a single canonical reading source avoids that
4	Build the priority-scoring function: $weight = f(\text{cluster size, category severity, time open, local AQI})$. Make weights configurable/visible — officers should be able to see <i>why</i> something ranks where it does, not trust a black box	Build <code>/clusters/priority</code> sorted endpoint	Build the priority queue list view, sorted, with visible score breakdown (this doubles as your explainability story)	Draft the "why not just fix SAMEER top-down" competitive answer — reuse Competitive Analysis Section 10/11
5	Design the recommendation prompt template: cluster category + severity + AQI trend + location → structured prompt variables	Wire <code>/clusters/{id}/recommend</code> → calls Gemini/Claude with the templated prompt, returns action text	Build the cluster detail panel: shows complaints in cluster, priority score, and a "Generate Recommendation" button	Cost/economic viability table (reuse Tools Proposal Section 5 — mostly unchanged, since the ML/backend stack is the same)
6	Add a lightweight AQI trend model (RF/XGBoost, per your ML doc) per zone — feeds "is this getting worse" into both priority score and the recommendation prompt	Build <code>/clusters/{id}/resolve</code> — logs action taken + before/after AQI snapshot	Build the "mark actioned" + before/after result UI	Sync checkpoint: confirm the full pipeline (ingest → cluster → prioritize → recommend → resolve) is realistic with 8 days left

Phase C — Integration & the Officer Workflow (Days 7-9)

Day	Focus
7	Full-team integration. Load synthetic complaints → watch them cluster on the map → confirm priority ordering makes intuitive sense → click a cluster → generate a recommendation → confirm it reads like something a real officer could act on, not generic AI filler.
8	

Day	Focus
	Close the loop: resolve a cluster, log a before/after AQI delta, and show it on a simple "impact log" view. This is your evidence-generation differentiator — the same one flagged in your Competitive Analysis as the industry-wide gap, now applied to the officer's own record of what worked.
9	Bug-fix day only. No new features without full-team agreement.

Phase D — Explainability, Polish & Positioning (Days 10-11)

Day	R1	R2	R3	R4
10	Sanity-check clustering/priority on edge cases (a single high-severity outlier complaint vs. a large low-severity cluster — does the ranking still make sense?)	Add a static fallback dataset so a live model/API hiccup never breaks the demo	UI polish: consistent styling, clear visual hierarchy (priority queue should be scannable in 5 seconds — that's the whole time-savings pitch)	Environmental impact slide: tie faster officer response to modeled exposure-time reduction (e.g., "X% faster triage → Y fewer exposure-days for the affected zone" — state the assumption plainly)
11	Validate recommendation prompt outputs across all categories — make sure "construction dust" and "biomass burning" don't get near-identical generic advice	Add basic error handling so a failed Gemini call degrades gracefully (show "recommendation pending," never crash)	Full click-through test of the entire officer workflow	Finalize the deck's core "time saved" number — this is the single stat judges will remember; make sure R1–R3 can defend it technically if asked

Phase E — Presentation Readiness (Days 12-13)

Day	Tasks (whole team)
12	Build the deck against the theme's rubric, reframed for the nodal-officer angle throughout: 1) Specific problem (manual triage bottleneck at the nodal-officer layer, not citizen awareness) 2) Innovative solution (cluster → prioritize → auto-recommend → track outcome) 3) Practical feasibility (working demo on real-ish synthetic data) 4) Implementation plan (14-day build → CPCB/SAMEER integration phase → multi-city scale) 5) Challenges (data quality/consistency, clustering accuracy at scale, officer trust in auto-recommendations) 6) Economic viability (cost table) 7) Environmental impact (Day 10 quantification)
13	Record a 2–3 min backup demo video. Full timed dry-run. Get outside feedback — specifically ask a non-team member "would a government officer actually trust this recommendation, or does it feel like a black box?" since that's your real adoption risk.

Phase F — Buffer & Submission (Day 14)

Day	Tasks
14	Final rehearsal only. No new code. Confirm deck exports correctly, video plays on another machine, GitHub repo accessible, and every team member can state the "time saved" number and defend the clustering/priority logic without notes.

4. Risk Table

Risk	Mitigation
Clustering looks unconvincing on random synthetic data	Deliberately seed 2–3 realistic hotspots when generating data (Day 2) — don't uniformly randomize
Auto-generated recommendations read as generic AI text	Spend real time on the prompt template (Day 5) — feed it structured cluster data, not just a category name
"Time saved" claim gets challenged by judges as unverified	State your baseline assumption explicitly in the deck (e.g., "assuming X minutes of manual triage per complaint today, based on [source/estimate]") rather than presenting it as measured fact
Officer-trust concern (black-box priority score)	Keep the priority score breakdown visible in the UI (Day 4) — this is also your explainability answer if judges ask
Live demo/API failure during judging	Static fallback dataset (Day 10) + recorded backup video (Day 13)

5. What "Done" Looks Like on Day 14

A judge watches: 500+ synthetic complaints load onto a Pune map → duplicate/nearby reports visibly collapse into a handful of clusters → the priority queue ranks them with a visible, explainable score → clicking the top cluster generates a specific, actionable recommendation in seconds → marking it resolved logs a measured AQI before/after. The pitch in one sentence: *what currently takes a nodal officer manual hours across scattered tickets, AirSense does in seconds with a transparent, defensible ranking.*